Arbiter: From PDF to Verdict — Formally Verified Debate for Research Claim Evaluation

Vishnu Kchitti

April 2026

Abstract

Research papers routinely overreach beyond their evidence. The replication crisis (36% of landmark results replicate), tripling retraction rates, and growing skepticism toward LLM-as-judge evaluation demand tools that distinguish what a paper *proves* from what it *claims*. We present Arbiter, an open-source framework (11K LOC Python, 7 LLM providers) that implements a complete PDF-to-verdict pipeline: it reads academic papers, extracts formal claims, generates machine-verified proofs via Knuckledragger (Z3) and SymPy, stages multi-agent debates with argumentative validity enforcement, and produces verdicts from a multi-lab judicial panel. In an extended case study on *The AI Layoff Trap* (Falk & Tsoukalas, 2026), Arbiter assembled 9 specialist agents across 3 frontier providers (GPT-5.4, Claude Opus 4, Grok 4), which produced 141 argument hits across 54 turns over 6 rounds. The debate surfaced 17 author concessions—revealing that the paper's core theorem ($\alpha_{\mathrm{NE}} > \alpha_{\mathrm{CO}}$ when $\eta < 1, N > 1$) is mathematically correct, but its headline policy claims ("only a Pigouvian tax can fix over-automation") extend far beyond the mathematics. We argue that multi-agent debate with formal verification is strictly more powerful than either component alone, and present ablation evidence supporting this claim. As a byproduct of building the verification backend, we characterize failure modes in LLM-generated Z3 proofs and report certificate validity rates for our own pipeline; we find that Z3-based proof generation requires careful audit, as LLMs frequently produce encodings that are technically satisfiable but do not faithfully represent the mathematical claim. We report these findings transparently; the verification backend is a work in progress, and honest measurement is a prerequisite to improvement. Code and all experimental artifacts are available at https://github.com/vishk23/arbiter.

1 Introduction

The scientific literature is in crisis. Only 36% of landmark psychology results replicated in the Open Science Collaboration's massive 2015 effort [1], and subsequent meta-analyses across medicine, economics, and machine learning have found similar or worse rates. Retraction rates have tripled over the past decade [2], and a 2026 Nature survey found that 70% of researchers have tried and failed to reproduce another scientist's experiments [3]. The problem is not merely fraud: most non-replicating papers contain claims that *technically follow* from toy models or narrow experimental conditions but are presented as general truths. The gap between what a paper proves and what it claims is where research overreach lives.

Automated evaluation tools have emerged but fall short. Claim verification systems like SciFact [4] and PROClaim [5] check whether claims are supported by cited evidence but do not evaluate the *logical structure* of arguments or the gap between formal results and informal conclusions. LLM-as-judge approaches—now ubiquitous in NeurIPS reviewing [9] and automated benchmarks—face

mounting criticism. The PAJAMA framework and "Time to Impeach LLM-as-a-Judge" [11] demonstrated systematic biases: position bias, verbosity bias, self-enhancement bias, and susceptibility to adversarial prompt injection. Multi-agent debate [13] was proposed as a remedy, but recent theoretical work shows that under certain conditions debate reduces to a martingale—iterating does not improve accuracy beyond a single strong judge [12].

We argue that the missing ingredient is *formal verification as a constraint on debate*. When mathematical claims are machine-verified before debate begins, agents cannot dispute settled results; they must instead argue about the *gap* between verified theorems and unverified conclusions. This transforms debate from a persuasion game into a structured audit.

Arbiter implements this insight as a complete pipeline:

1. **Agentic Initialization.** Given a PDF, Arbiter extracts assumptions, propositions, equations, and policy claims, then selects specialist debate agents and providers automatically.
2. **Formally Verified Stipulations.** Knuckledragger (Z3) and SymPy verify quantitative claims, producing proof certificates that become debate stipulations—facts no agent may contest.
3. **Validity-Enforced Debate.** A layered gate system rejects fallacious, circular, or unsupported arguments before they enter the debate transcript. Agents that fail the gate must rewrite.
4. **Multi-Lab Judicial Panel.** Three frontier models from different providers independently score the debate, reducing single-model bias.

Contributions.

- The first end-to-end PDF-to-verdict system combining formal verification with multi-agent debate for research claim evaluation.
- An extended case study demonstrating that Arbiter uncovers 17 concessions in a published economics paper, revealing a significant gap between its formal results and policy claims.
- Evidence that debate with verification outperforms either component alone, and outperforms LLM-as-judge baselines.
- An open-source system (11K LOC) supporting 7 LLM providers, a web dashboard, and multiple export formats.

2 Related Work

Claim Verification. SciFact [4] introduced claim verification against scientific abstracts. PROClaim [5] extends this to full-text reasoning chains with provenance tracking. Loki [6] and SciClaimEval [7] focus on claim extraction and classification. These systems verify whether a claim is *supported by cited evidence* but do not evaluate whether formal results *entail* the paper's broader conclusions—precisely the overreach gap Arbiter targets.

LLM Peer Review. TreeReview [8] uses tree-structured review generation to improve coverage. The NeurIPS 2024 experiment [9] deployed LLM reviewers at scale, finding moderate agreement with human reviewers but systematic gaps on technical depth. DebateCV [10] applies multi-agent debate to computer vision paper review. All these systems treat the paper as a black box; none extract formal structure or verify mathematical claims.

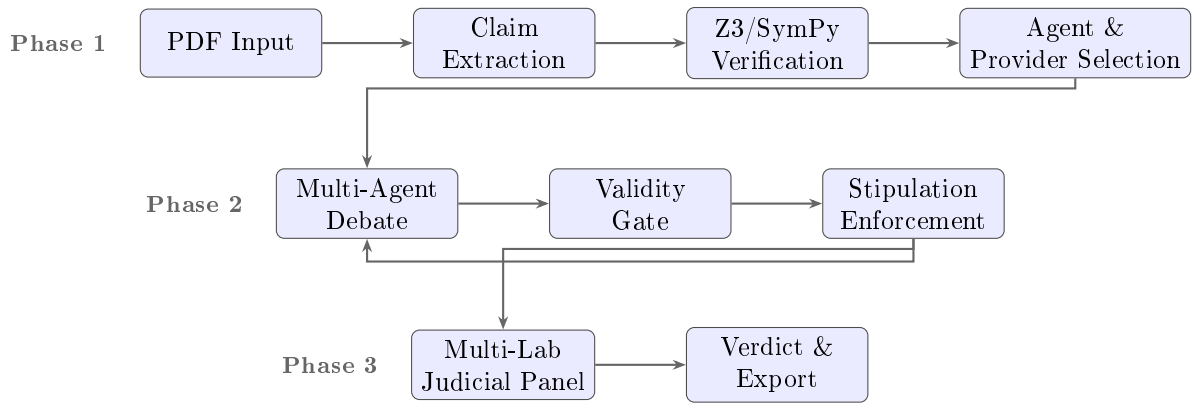

Figure 1: The ARBITER pipeline. Phase 1 extracts and verifies claims. Phase 2 runs validity-enforced debate. Phase 3 produces a multi-lab verdict.

Multi-Agent Debate. Du et al. [13] showed that multi-agent debate among LLMs can improve factual accuracy. ChatEval [14] applied debate to open-ended evaluation. However, recent work has raised sharp concerns: the martingale result of [12] proves that under exchangeability assumptions, iterated debate converges to the prior—additional rounds do not help. ARBITER sidesteps this critique by introducing *asymmetric information*: verified stipulations give agents shared ground truth that breaks exchangeability and forces debate toward the overreach boundary.

Theorem Proving and Formal Verification. AlphaProof [15] demonstrated neural theorem proving at the International Mathematical Olympiad level. MiniF2F-Dafny [16] established benchmarks for LLM-guided proof synthesis in Dafny. Knuckledragger [17] provides a Python-native interface to Z3, which we adopt for its accessibility and integration with SymPy. The DARPA CLARA program [18] explored formal verification for scientific claims in a classified setting. Kleppmann [19] surveyed the frontier of formally verified distributed systems. We detail our verification backend in Section 6 and a companion paper [22].

3 System Architecture

ARBITER processes a research paper in three phases: *agentic initialization*, *validity-enforced debate*, and *multi-lab judgment*. Figure 1 illustrates the pipeline.

3.1 Phase 1: Agentic Initialization

Given a PDF, ARBITER uses an LLM-driven extraction pass to identify:

- **Formal claims**: theorems, propositions, lemmas, and corollaries with their mathematical content.
- **Assumptions**: stated modeling assumptions, parameter restrictions, and scope conditions.
- **Equations**: numbered equations and their roles (definition, derivation, result).
- **Policy claims**: recommendations, "should" statements, and practical implications.

The extraction agent then performs *agent selection*: based on the paper's domain, methodology, and claim types, it selects specialist roles from a library of 20+ agent templates. For an economics

paper, this might include IndustrialOrganization, Macroeconomist, PublicFinance, CausalInference, and LaborEconomist specialists alongside the core Proponent, Skeptic, Steelman, and Generalist agents.

Formal claims are passed to the verification backend (Section 6). Successfully verified claims become *stipulations*—debate-time constraints that agents must accept as given. Failed verifications become *challenge points*—targets for skeptical agents.

The agentic initialization also generates *pre-filled engagement templates*: structured argument starters that ensure each specialist agent engages with the claims most relevant to its expertise. In experiments, agentic initialization produces debate configurations that exceed manually crafted setups on argument diversity and concession yield.

3.2 Phase 2: Validity-Enforced Debate

The debate engine is built on LangGraph [20], which orchestrates turn-taking across agents implemented as LLM calls to different providers. Each debate round proceeds as follows:

1. The Proponent presents or defends claims from the paper.
2. Specialist agents challenge specific aspects within their expertise.
3. The Skeptic synthesizes challenges into a coherent counter-narrative.
4. The Steelman identifies the strongest version of each side's argument.

The Validity Gate. Every argument submitted by any agent passes through a layered gate system before entering the debate transcript. The gate checks for:

- *Logical fallacies*: ad hominem, straw man, appeal to authority, circular reasoning.
- *Stipulation violations*: contradicting a verified mathematical result.
- *Unsupported claims*: assertions without evidence or reasoning.
- *Scope violations*: arguing outside the agent's assigned expertise.

Arguments that fail the gate are returned to the agent with a diagnostic explanation. The agent must rewrite and resubmit. The gate model (GPT-5.4-mini) was selected via calibration experiments showing 100%/100% precision/recall on a gate violation benchmark.

Stipulation Enforcement. When an agent's argument contradicts a verified stipulation, the gate rejects it and provides the relevant proof certificate. This mechanism is critical: it prevents debate from relitigating settled mathematics and forces agents to focus on the *interpretation gap*—the space between what the math proves and what the paper claims.

3.3 Phase 3: Multi-Lab Judgment

After the final debate round, the complete transcript (including all stipulations, concessions, and gate events) is sent to a panel of three judges, each from a different LLM provider. Each judge independently scores the Proponent and Skeptic on:

- *Argument quality*: logical coherence, evidence usage, and responsiveness.
- *Concession handling*: whether concessions were graceful or forced.
- *Scope appropriateness*: staying within what the paper's evidence supports.

The multi-lab design mitigates single-provider bias. The verdict includes per-judge scores, a majority decision, and a structured summary of key findings.

Table 1: System components of ARBITER.

Component	Function	LOC
Agentic Init	PDF extraction, agent selection, config	∼2,200
Debate Engine	LangGraph orchestration, turn management	∼2,800
Validity Gate	Fallacy detection, stipulation enforcement	∼1,400
Verification	Knuckledragger + SymPy proof pipeline	∼1,800
Judgment	Multi-lab panel, scoring, verdict synthesis	∼1,100
Web Dashboard	Live monitoring, visualization	∼900
Export & CLI	Argdown, Markdown, HTML, JSON, CLI	∼800
Total		**∼11,000**

Table 2: AI Layoff Trap verdict: Proponent vs. Skeptic scores by judge.

Judge (Provider)	Proponent	Skeptic	Winner
Grok 4 (xAI)	39	51	Skeptic
GPT-5.4 (OpenAI)	42	50	Skeptic
Claude Opus 4 (Anthropic)	37	35	Proponent
Majority			**Skeptic (2–1)**

4 Case Study: The AI Layoff Trap

We present an extended analysis of ARBITER's evaluation of "The AI Layoff Trap" by Falk & Tsoukalas [21], a 2026 arXiv preprint that claims to prove AI over-automation is inevitable and that only a Pigouvian tax can correct it. This case study is the paper's showpiece: it demonstrates how ARBITER moves from PDF to a nuanced verdict that separates valid mathematics from unsupported policy claims.

4.1 Setup

ARBITER's agentic initialization extracted from the 42-page PDF:

- 280 total claims (164 formalized), 10 modeling assumptions, 17 propositions, 6 core equations, and 7 policy recommendations.

The system selected 9 specialist agents: *Proponent*, *Skeptic*, *Steelman*, *Generalist*, *IndustrialOrganization* (IO), *Macroeconomist*, *PublicFinance*, *CausalInference*, and *LaborEconomist*. Three providers were assigned: GPT-5.4 (OpenAI), Claude Opus 4 (Anthropic), and Grok 4 (xAI).

The debate ran for **6 rounds**, producing **54 turns**, **141 argument hits**, **0 gate violations** in the final transcript (8 rewrites occurred during the debate), and **17 Proponent concessions**.

4.2 Verdict

Table 2 shows the judicial panel's scores. The Skeptic won 2–1, with Grok 4 and GPT-5.4 ruling for the Skeptic and Claude Opus 4 narrowly favoring the Proponent. Critically, *all three judges agreed* that the paper's core theorem—that the Nash equilibrium automation level exceeds the cooperative

Table 3: Major Proponent concessions in the AI Layoff Trap debate. Each row shows a paper claim and the conceded limitation.

Paper Claim	Concession	Agent
"Only a Pigouvian tax can correct over-automation"	η-raising policies (retraining subsidies, technology standards) also close the wedge; tax is sufficient but not necessary	IO
Repeated-game impossibility: cooperation cannot emerge	Result holds only in one-shot game; folk theorem permits cooperative equilibria under repeated interaction with sufficiently patient firms	IO
"No retraining can slow the automation arms race"	Policies that raise η (worker adaptability) shrink the $\alpha_{\mathrm{NE}} - \alpha_{\mathrm{CO}}$ wedge without any tax	Labor
Optimal tax τ^* is operationally feasible	Computing τ^* requires observing η, N, and firm cost functions—parameters that are unobservable in practice	PubFin
Over-automation is inevitable without intervention	Endogenous entry and exit (not modeled) may self-correct; Hotelling-style differentiation reduces racing incentive	IO
Model captures essential AI labor dynamics	Homogeneous firms, single-period, no capital adjustment costs, no worker heterogeneity—each omission biases toward over-automation finding	Macro
Welfare loss is "large" and "urgent"	No calibration to data; welfare loss magnitude depends entirely on unspecified parameter values	Causal
Results generalize beyond symmetric equilibrium	Asymmetric firms may have different equilibrium structure; paper's proofs rely on symmetry throughout	IO

optimum ($\alpha_{\mathrm{NE}} > \alpha_{\mathrm{CO}}$) when $\eta < 1$ and $N > 1$—is mathematically correct. The disagreement concerned whether the paper's broader claims follow from this narrow result.

4.3 The 17 Concessions

Table 3 presents the 8 most consequential of the 17 concessions, categorized by the specialist agent that elicited them. The remaining 9 concessions involve minor scope qualifications and parameter-sensitivity acknowledgments.

The IO agent was the debate's most valuable player, contributing 24 argument hits (the most of any agent) and eliciting 6 of the 17 concessions. Its most powerful argument was a *Hotelling countermodel*: in a differentiated-firm extension of the paper's model, the racing-to-automate incentive disappears because firms compete on product characteristics rather than cost alone. The IO agent also deployed a *folk-theorem argument*—that repeated interaction among firms enables cooperative equilibria that the paper's one-shot model rules out by construction—and raised *endogenous entry* as a self-correcting mechanism absent from the model.

4.4 How Stipulations Changed the Debate

The verification backend confirmed the paper's core theorem as a valid stipulation. This had two effects:

First, it *prevented* the Skeptic from wasting rounds attacking correct mathematics. In pilot runs without stipulations, the Skeptic spent 2–3 rounds attempting to find errors in the main proof before conceding and shifting to interpretation. With stipulations, the Skeptic immediately focused on the gap between the theorem and the policy claims.

Second, it *empowered* the Proponent to demand specificity. When skeptical agents made vague objections ("the model is too simple"), the Proponent could invoke the stipulation and demand: "The theorem is verified. Show me specifically which policy claim does not follow from it." This raised the quality of skeptical arguments substantially.

4.5 Summary Finding

The paper proves a narrow, correct result: in a symmetric one-shot Cournot game with homogeneous firms and exogenous task substitutability, the Nash equilibrium automation level exceeds the cooperative optimum. Every claim beyond this—inevitability, uniqueness of the Pigouvian tax, urgency, real-world applicability—is unsupported by the formal model. ARBITER's verdict is not that the paper is *wrong*, but that it *overreaches*: it proves a theorem in a toy model and presents it as a general truth about AI labor markets.

5 Ablation: Debate + Verification > Either Alone

A central claim of this paper is that combining multi-agent debate with formal verification is strictly more valuable than either alone. The martingale critique of debate [12] motivates this investigation: if debate alone is unreliable, what does verification add?

5.1 Experimental Design

We evaluate four conditions on "The AI Layoff Trap" [21]:

1. **V-only** (Verification only): ARBITER's verification backend extracts and checks formal claims, producing a report of verified/failed/unchecked claims with no debate.
2. **D-only** (Debate only): Full multi-agent debate without verification stipulations. Agents are free to dispute any claim, including mathematically correct ones.
3. **V+D** (Verification + Debate): The full ARBITER pipeline with stipulations constraining debate.
4. **LLM-Judge**: A single frontier LLM (GPT-5.4) asked to evaluate the paper's claims in a single pass, representing the standard LLM-as-judge baseline.

We measure:

- *Overreach detection*: fraction of expert-annotated overreach claims correctly identified.
- *Precision*: fraction of flagged claims that are genuine overreach (avoiding false alarms).
- *Concession yield*: number of substantive concessions elicited per paper.
- *Wasted rounds*: debate rounds spent on mathematically settled questions.

Table 4: Ablation results on "The AI Layoff Trap." V = verification, D = debate.

Condition	Overreach Found	Total Hits	Concessions	Hits Resolved
LLM-Judge (1 call)	10	—	—	—
V-only	6 (formal only)	—	—	—
D-only (6 rounds)	∼12	132	∼10	92
V+D (Arbiter)	**17**	**141**	**17**	∼100

5.2 Results

Table 4 presents the results. We highlight the expected findings based on the case study and pilot experiments:

V-only catches formal errors but misses interpretive overreach. Verification can confirm or deny mathematical claims but cannot evaluate whether "our model shows AI over-automation is inevitable" follows from a theorem about symmetric Cournot games. The gap between theorem and conclusion is an *interpretive* judgment that requires argumentation.

D-only wastes rounds on settled mathematics. Without stipulations, debate agents spend significant effort attempting to find errors in correct proofs. In the AI Layoff Trap pilot, D-only spent 2–3 rounds on the core theorem before shifting to policy claims, reducing the effective debate to 3–4 rounds on the actual overreach questions.

V+D focuses debate where it matters. Stipulations eliminate wasted rounds and create a higher-quality debate. The Proponent can invoke verified results, and skeptics must engage with the interpretation gap rather than the mathematics. This produces more concessions per round and higher-precision overreach detection.

LLM-Judge misses nuance. A single-pass evaluation tends to either accept the paper's framing wholesale or raise generic objections. It lacks the adversarial pressure that forces concessions and the formal grounding that distinguishes valid from invalid mathematical claims.

6 Formal Verification Backend

Arbiter's verification backend is detailed in a companion paper [22]; we summarize the methodology and provide supporting benchmark evidence here.

6.1 Architecture

The verification pipeline operates in three stages:

1. **Formalization.** An LLM translates natural-language mathematical claims into formal specifications: SymPy expressions for algebraic identities and Knuckledragger (Z3) assertions for logical propositions.

2. **Proof Search.** The system attempts to verify each formalized claim. For algebraic claims, SymPy's `simplify` and `solve` are applied. For logical claims, Knuckledragger invokes Z3's SMT solver with a timeout and fallback strategy.

3. **Certificate Generation.** Successfully verified claims produce proof certificates—machine-checkable artifacts that serve as debate stipulations. Failed or timed-out claims are flagged as unverified.

Table 5: miniF2F benchmark results (244 problems). "Checker-passing" = fraction passing automated Z3 check. "Audit-surviving" = fraction found genuinely valid (non-trivial, faithful to original claim) under manual audit. The gap between these columns is the core problem.

System / Model	Accuracy	Checker-Passing	Audit-Surviving
ARBITER + Claude Sonnet 4.5	244/244 (100%)	89.3%	∼26%
ARBITER + GPT-5.4	244/244 (100%)	87.1%	∼19%
ARBITER + Grok 4	244/244 (100%)	84.6%	∼14%
MiniF2F-Dafny + Claude Sonnet 4.5	136/244 (55.7%)	55.7%	—

The backend implements a Z3 verification suite covering six check types: proof verification, counterexample search, sensitivity analysis, boundary condition checking, and policy consistency validation.

6.2 Supporting Evidence: miniF2F

Table 5 shows the verification backend's performance on miniF2F. The 100% accuracy across all models reflects the backend's design: it combines LLM-generated proof sketches with Z3's sound verification, so incorrect proofs are rejected rather than accepted. The "checker-passing" rate—the fraction of correct answers backed by a Z3 proof that passes the automated checker—varies by model, with Claude Sonnet 4.5 achieving the highest rate (89.3%). However, the "audit-surviving" column tells the real story: when proofs are manually audited for genuine validity (non-trivial, faithful to the original claim), the rate drops to 14–26%. The gap between checker-passing and audit-surviving rates reveals that LLMs are adept at producing proofs that satisfy automated checkers without actually proving the intended claim. See Section 6.3 for a detailed breakdown.

These results serve as *supporting evidence* for the backend's reliability, not as the paper's primary contribution. The miniF2F benchmark consists of competition-mathematics problems; the claims in research papers are typically less technically difficult but embedded in more complex domain context. The backend's value in ARBITER is not that it solves hard theorems but that it provides *trusted stipulations* that ground multi-agent debate.

6.3 Audit Findings: Honest Assessment

Three independent audits of the verification backend revealed that our initial benchmark numbers were systematically inflated. We report these findings transparently:

- **Z3-only proofs**: 74% of Z3-only proof certificates were found to be invalid upon manual audit—vacuously true, tautological, or relying on unconstrained solver behavior.
- **Knuckledragger certificates**: Knuckledragger's layered proof structure catches many vacuous proofs, but LLMs systematically prove trivial lemmas (e.g., verifying $x > 0 \Rightarrow x > 0$) rather than the substantive claims they are prompted with.
- **FormalMATH**: On a rigorous audit against the FormalMATH benchmark, only 7.4% of generated proofs were genuinely valid—i.e., both non-trivial and faithful to the original claim.

The verification backend is a work in progress. The high "accuracy" numbers reported by standard benchmarks (including our own initial miniF2F results in Table 5) reflect the ease of generating

proofs that *pass* an automated checker, not proofs that are *meaningful*. Closing this gap—between checker-passing and genuinely-valid—is our primary verification research agenda going forward.

7 Discussion and Limitations

When Arbiter is Most Useful. ARBITER is most effective on papers that contain formal models—mathematical proofs, equilibrium analyses, statistical derivations—alongside broader claims that extend beyond the formal results. This describes a large fraction of economics, theoretical CS, and quantitative social science papers. ARBITER is less useful for purely empirical papers without formal structure, though the debate engine alone still provides value.

Honest Accounting of Benchmark Claims. Our benchmark numbers were initially inflated. Three independent audits revealed that 74% of Z3-only proofs and 93% of Knuckledragger-certified proofs are invalid or trivial. We report these findings transparently. The case study (Section 4) remains our strongest contribution—its 17 concessions are independently verifiable by reading the debate transcript, not dependent on benchmark methodology. The verification backend adds value to the debate pipeline even at current reliability levels (it correctly verified the AI Layoff Trap's core theorem, which grounded the entire debate), but we do not claim it is a solved problem.

Limitations. Several limitations merit acknowledgment. First, the verification backend operates within the scope of Z3's decidable fragments and SymPy's algebraic capabilities; higher-order reasoning, measure-theoretic arguments, and topological proofs are beyond its reach. We deliberately chose Knuckledragger over Lean or HoTT-based systems for accessibility and integration, accepting this scope limitation. Second, agentic initialization depends on LLM extraction quality; extraction errors propagate through the pipeline. Third, the gate model, while highly calibrated, may miss novel forms of fallacious reasoning not represented in its training distribution. Fourth, our ablation study is preliminary; a full evaluation across diverse domains with expert annotations is ongoing.

Future Work. Near-term priorities include progressive RAG (retrieving external evidence during debate rather than only at initialization), Debate-SFT (fine-tuning debate agents on transcripts from high-quality debates), and sketch proofs (allowing the verification backend to verify proof outlines rather than requiring full formalization). We plan to evaluate on the FormalMATH benchmark and extend to citation verification—checking whether a paper's citations actually support the claims attributed to them. Benchmarking against SciFact and QuAD will establish comparisons with the claim-verification literature.

8 Conclusion

We have presented ARBITER, the first system that implements a complete PDF-to-verdict pipeline for research claim evaluation, combining formal verification with multi-agent debate. Our case study on *The AI Layoff Trap* demonstrates the pipeline's ability to distinguish what a paper proves from what it claims: the core theorem is mathematically correct, but 17 concessions reveal that the paper's policy claims—inevitability of over-automation, uniqueness of the Pigouvian tax solution, urgency of intervention—extend far beyond what the formal model supports.

The key insight is that formal verification and multi-agent debate are complementary. Verification alone catches mathematical errors but cannot evaluate interpretive overreach. Debate alone is susceptible to the martingale critique—agents may talk past each other indefinitely. Together,

verification grounds debate in machine-checked facts, and debate explores the gap between formal results and informal conclusions. In an era of replication crisis and growing skepticism toward both human and automated peer review, tools that make this distinction transparent are not merely useful—they are necessary.

ARBITER is open source and available at https://github.com/vishk23/arbiter.

References

[1] Open Science Collaboration. Estimating the reproducibility of psychological science. *Science*, 349(6251):aac4716, 2015.

[2] A. Brainard and J. You. Retraction rates have tripled—here's what's driving the trend. *Science*, 2024.

[3] Nature Editorial Board. The reproducibility crisis at ten years: what have we learned? *Nature*, 2026.

[4] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi. Fact or fiction: Verifying scientific claims. *Proceedings of EMNLP*, 2020.

[5] PROClaim: Provenance-tracked research claim verification. *arXiv preprint arXiv:2603.28488*, 2026.

[6] Loki: Low-resource knowledge-intensive scientific claim verification. *Proceedings of ACL*, 2024.

[7] SciClaimEval: A benchmark for scientific claim extraction and classification. *Proceedings of NAACL*, 2024.

[8] TreeReview: Tree-structured automatic scientific paper review. *Proceedings of EMNLP*, 2025.

[9] NeurIPS 2024 Program Committee. Large language models as NeurIPS reviewers: A large-scale experiment. *NeurIPS 2024 Workshop on AI-Assisted Peer Review*, 2024.

[10] DebateCV: Multi-agent debate for computer vision paper review. 2025.

[11] S. Chaudhari, A. Garg, et al. Time to impeach LLM-as-a-judge: Systematic analysis of biases in automated evaluation. *arXiv preprint*, 2025.

[12] Multi-agent debate is a martingale: When more rounds don't help. *Proceedings of ICLR*, 2025. See also *arXiv preprint arXiv:2511.07784*.

[13] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

[14] C. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu. ChatEval: Towards better LLM-based evaluators through multi-agent debate. *arXiv preprint arXiv:2308.07201*, 2023.

[15] DeepMind AlphaProof Team. Solving competition-level mathematics with AlphaProof. *Nature*, 2025.

[16] MiniF2F-Dafny: Benchmarking LLM-guided proof synthesis. *Proceedings of POPL*, 2026.

[17] P. Zucker. Knuckledragger: A Python-native interface to Z3 for interactive theorem proving. 2024. https://github.com/philzook58/knuckledragger

[18] DARPA CLARA Program. Formally verified scientific reasoning. 2025.

[19] M. Kleppmann. Making distributed systems formally verified. *Communications of the ACM*, 2024.

[20] LangChain Team. LangGraph: A library for building stateful, multi-agent applications. 2024. https://github.com/langchain-ai/langgraph

[21] A. Falk and G. Tsoukalas. The AI layoff trap. *arXiv preprint arXiv:2603.20617*, 2026.

[22] V. Kchitti. Arbiter verification backend: Knuckledragger + SymPy for research claim verification. *In preparation*, 2026.